

Algorytmy i struktury danych, Teleinformatyka, I rok

Raport z laboratorium nr: A6

Imię i nazwisko studenta: Paweł Kowalcze

nr indeksu: 420588

1. W pole poniżej wklej najważniejszy (według Ciebie) fragment kodu źródłowego z zajęć (maksymalnie 15 linii).

```
def fits_in_knapsack(knapsack, width, height, x, y):
    if x + width > len(knapsack) or y + height > len(knapsack):
        return False
    for i in range(width):
        for j in range(height):
            if knapsack[x + i][y + j] != 0:
                return False
    return True
```

Ten fragment kodu jest dla mnie najważniejszy, ponieważ pisząc go nauczyłem się w jaki sposób można sprawdzać, czy jakiś obiekt posiadający konkretne rozmiary jest w stanie zmieścić się wewnątrz drugiego obiektu.

2. Podsumuj wyniki uzyskane podczas wykonywania ćwiczenia. Co ciekawego zauważyłeś? Czego się nauczyłeś? Jeśli instrukcja zawierała pytania, odpowiedz na nie. Do sprawozdania możesz dodać wykresy, tabele jeśli jest taka potrzeba.

Algorytmy wykorzystane przeze mnie to algorytm zachłanny, First-Fit Decreasing oraz Best-Fit Decreasing.

Algorytm zachłanny działa w taki sposób, że sortuje przedmioty na podstawie ich wartości a następnie bierze pod uwagę każdy przedmiot, od największej do najmniejszej wartości, i próbuje go umieścić w plecaku (przy tym zwraca uwagę na wymiary przedmiotu i decyduje czy zmieści się w środku). Rozwiązanie tego algorytmu nie jest tak optymalne jak pozostałych dwóch algorytmów.

Algorytm First-Fit Decreasing jest heurystyką służącą do rozwiązywania problemów związanych z pakowaniem pojemników. Polega on na sortowaniu przedmiotów według rozmiaru (lub innej miary, takiej jak wartości) w kolejności malejącej, a następnie umieszczaniu każdego przedmiotu na pierwszej dostępnej pozycji, która może go pomieścić. W tym przypadku przedmioty są sortowane od największej do najmniejszej wartości.

Algorytm Best-Fit Decreasing jest kolejnym algorytmem heurystycznym do rozwiązywania problemów związanych z pakowaniem pojemników. Umieszcza on każdy element w pozycji, która pozostawia najmniej wolnej przestrzeni. Jest on bardziej wyrafinowany niż First-Fit Decreasing, ponieważ stara się zminimalizować pozostałą przestrzeń, co może prowadzić do lepszego pakowania i wyższej wartości całkowitej. Wymaga on jednak więcej obliczeń, ponieważ ocenia więcej potencjalnych miejsc dla każdego elementu.

Wyniki:

Algorytm zachłanny:

[[1 2 3 3 3 3 3 4 4 4 4 5 6 6 6 6 6 7 7 7]]

[1 2 8 8 8 9 9 4 4 4 4 5 6 6 6 6 6 7 7 7]

[11 11 8 8 8 9 9 10 10 10 14 5 6 6 6 6 6 7 7 7]

[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]

```
[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]
```

```
[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]
```

```
[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]
```

```
[11 11 0 0 0 9 9 10 10 10 0 0 12 12 12 15 15 15 16 0]
```

[11 11 0 0 0 9 9 0 0 0 0 0 12 12 12 15 15 15 16 0]

[11 11 0 0 0 9 9 0 0 0 0 0 12 12 12 15 15 15 16 0]

[13 13 13 13 13 13 13 13 13 13 0 0 0 0 0 15 15 15 16 0]

[13 13 13 13 13 13 13 13 13 13 13 17 17 17 17 17 17 0 0 16 0]

[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]

```
[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]
```

```
[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]
```

[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]

```
[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]
```

```
[18 18 18 18 18 18 18 18 18 18 19 19 19 19 19 19 19 19 0 0]
```

[18 18 18 18 18 18 18 18 18 18 19 19 19 19 19 19 19 19 0 0]

[0 0 0 0 0 0 0 0 0 19 19 19 19 19 19 19 19 0 0]]

Greedy, 20, TIME: 0.008520841598510742

Total value collected: 131

Max possible value: 161

[[1 2 3 ... 44 44 46]

[48 49 50 ... 44 44 46]

[48 49 50 ... 44 44 46]

...

[385 385 385 ... 388 388 388]

[385 385 385 ... 388 388 388]

[0 0 0 ... 388 388 388]]

Greedy, 100, TIME: 1.837777853012085

Total value collected: 2388

Max possible value: 2653

[[1 2 3 ... 324 326 330]

[325 325 325 ... 324 326 330]

[503 503 503 ... 654 654 330]

...

[9833 9833 9833 ... 9863 0 0]

[9833 9833 9833 ... 9863 0 0]

[0 0 0 ... 0 0 0]]

Greedy, 500, TIME: 1116.6531887054443

Total value collected: 62240

Max possible value: 68634

Algorytm FFD:

[[1 2 3 3 3 3 3 4 4 4 5 6 6 6 6 7 7 7]

[1 2 8 8 8 9 9 4 4 4 4 5 6 6 6 6 7 7 7]

[11 11 8 8 8 9 9 10 10 10 14 5 6 6 6 6 7 7 7]

[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]

[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]

[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]

[11 11 8 8 8 9 9 10 10 10 14 5 12 12 12 15 15 15 16 0]

[11 11 0 0 0 9 9 10 10 10 0 0 12 12 12 15 15 15 16 0]

[11 11 0 0 0 9 9 0 0 0 0 0 12 12 12 15 15 15 16 0]

[11 11 0 0 0 9 9 0 0 0 0 0 12 12 12 15 15 15 16 0]

[13 13 13 13 13 13 13 13 13 13 0 0 0 0 0 15 15 15 16 0]

[13 13 13 13 13 13 13 13 13 13 17 17 17 17 17 17 0 0 16 0]

[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]

[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]

[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]

[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]

[18 18 18 18 18 18 18 18 18 0 17 17 17 17 17 17 0 0 0 0]

[18 18 18 18 18 18 18 18 18 23 23 23 23 23 23 23 23 0 0]

[18 18 18 18 18 18 18 18 18 23 23 23 23 23 23 23 23 0 0]

[0 0 0 0 0 0 0 0 0 23 23 23 23 23 23 23 23 0 0]]

First-Fit Decreasing, 20, TIME: 0.004974842071533203

Total value collected: 131

Max possible value: 161

[[1 2 3 ... 44 44 46]

[48 49 50 ... 44 44 46]

[48 49 50 ... 44 44 46]

...

[385 385 385 ... 387 387 387]

[385 385 385 ... 387 387 387]

[0 0 0 ... 0 0 0]]

First-Fit Decreasing, 100, TIME: 2.2403013706207275

Total value collected: 2386

Max possible value: 2653

[[1 2 3 ... 324 326 330]

[325 325 325 ... 324 326 330]

[503 503 503 ... 654 654 330]

...

[9852 9852 9852 ... 0 0 0]

[9852 9852 9852 ... 10643 10643 10643]

[0 0 0 ... 10643 10643 10643]]

First-Fit Decreasing, 500, TIME: 1707.90345621109

Total value collected: 62240

Max possible value: 68634

[[1 2 3 ... 769 769 770]

[771 772 773 ... 1220 1243 770]

[771 772 1222 ... 1598 1243 770]

...

[39690 39690 39690 ... 39984 39984 39984]

[39690 39690 39690 ... 39984 39984 39984]

[0 0 0 ... 39984 39984 39984]]

First-Fit Decreasing, 1000, TIME: 28770.522980451584

Total value collected: 249441

Max possible value: 274120

Algorytm BFD:

```
[[ 0 0 18 18 18 18 18 18 18 18 18 0 0 0 0 0 0 0 0]
[ 0 0 18 18 18 18 18 18 18 18 18 18 19 19 19 19 19 19 19]
[ 0 0 18 18 18 18 18 18 18 18 18 18 19 19 19 19 19 19 19]
[ 0 0 18 18 18 18 18 18 18 18 18 18 19 19 19 19 19 19 19]
[ 0 0 18 18 18 18 18 18 18 18 18 18 17 17 17 17 17 17 16 0 0]
[ 0 0 18 18 18 18 18 18 18 18 18 18 17 17 17 17 17 17 16 0 0]
[ 0 0 18 18 18 18 18 18 18 18 18 18 17 17 17 17 17 17 16 0 0]
[ 0 0 0 13 13 0 0 0 0 0 0 17 17 17 17 17 17 16 0 0]
[ 0 0 0 13 13 0 0 0 0 0 0 17 17 17 17 17 17 16 0 0]
[15 15 15 13 13 0 0 0 0 0 0 17 17 17 17 17 17 16 0 0]
[15 15 15 13 13 12 12 12 0 0 0 0 9 9 0 0 16 11 11]
[15 15 15 13 13 12 12 12 0 0 0 0 9 9 0 0 16 11 11]
[15 15 15 13 13 12 12 12 0 0 10 10 10 9 9 0 0 16 11 11]
[15 15 15 13 13 12 12 12 5 14 10 10 10 9 9 8 8 8 11 11]
[15 15 15 13 13 12 12 12 5 14 10 10 10 9 9 8 8 8 11 11]
[15 15 15 13 13 12 12 12 5 14 10 10 10 9 9 8 8 8 11 11]
[15 15 15 13 13 12 12 12 5 14 10 10 10 9 9 8 8 8 11 11]
[15 15 15 13 13 12 12 12 5 14 10 10 10 9 9 8 8 8 11 11]
[ 7 7 7 6 6 6 6 6 5 14 10 10 10 9 9 8 8 8 11 11]
[ 7 7 7 6 6 6 6 6 5 4 4 4 4 9 9 8 8 8 2 1]
[ 7 7 7 6 6 6 6 6 5 4 4 4 4 3 3 3 3 3 2 1]]
```

Best-Fit Decreasing, 20, TIME: 0.047669410705566406

Total value collected: 131

Max possible value: 161

```
[[397 397 397 ... 0 0 0]
```

[397 397 397 ... 0 0 0]

[397 397 397 ... 0 0 0]

...

[44 44 43 ... 47 46 45]

[44 44 43 ... 47 46 45]

[44 44 43 ... 3 2 1]]

Best-Fit Decreasing, 100, TIME: 30.65916609764099

Total value collected: 2395

Max possible value: 2653

[[0 0 0 ... 0 0 0]

[9896 9896 9896 ... 9825 9825 9825]

[9896 9896 9896 ... 9825 9825 9825]

...

[736 325 734 ... 560 560 559]

[326 325 324 ... 328 327 327]

[326 325 324 ... 3 2 1]]

Best-Fit Decreasing, 500, TIME: 25168.332879304886

Total value collected: 62377

Max possible value: 68634

Dla algorytmu zachłannego i FFD wyniki pod względem wartości są identyczne lecz czas wykonania dla FFD jest dłuższy. Algorytm BFD uzyskał najlepsze wyniki pod względem wartości kosztem czasu wykonania algorytmu, który był zdecydowanie dłuższy od poprzednich.

Dzięki temu zadaniu nauczyłem się, jak działają algorytmy aproksymacyjne oraz w jaki sposób można je zaimplementować. Zauważyłem, mają one istotny wpływ na uzyskane wyniki, lecz wykonują się znacznie dłużej.